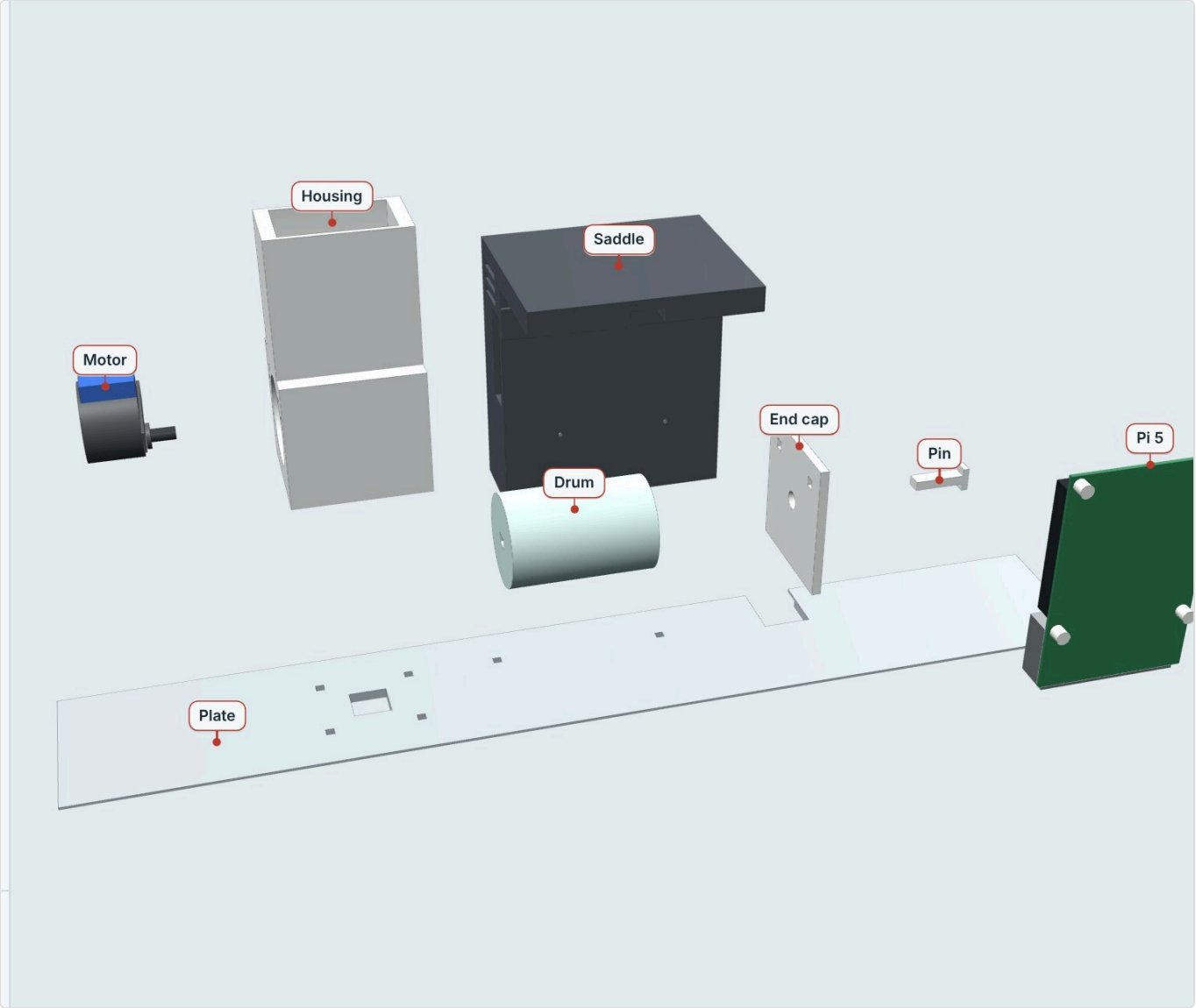
Smart Aquarium: Automatic Fish Feeder and Vision Rig

A research aquarium at the University of Rhode Island's CYPHER Lab, built around a Raspberry Pi 5 that watches the fish with a USB camera and YOLO, and a second Raspberry Pi 5 that feeds them with a 3D-printed stepper-driven drum feeder. Every part of the feeder was designed parametrically in code (KCL in Zoo Design Studio), geometry-checked in Python before printing, printed in PETG at URI's i² Studio, and assembled and tested by hand against the real hardware.



The repository holds the CAD source, the Pi control software, the wiring map, the build guide, and the lessons learned along the way.

Contents

1. What the system does
2. Hardware overview
3. The feeder mechanism
4. 3D design and printing
5. Electronics and wiring
6. Software
7. Assembly guide
8. Camera mount (next build)
9. Machine learning
10. What went wrong and what fixed it
11. Status and roadmap
12. Repository layout

What the system does

The tank is a LANDEN 60P rimless aquarium (600 x 300 x 360 mm, 6 mm glass, roughly 17 gallons) stocked with eight cherry barbs. Nobody is in the lab on weekends, so the fish needed a feeder that could be triggered remotely and trusted to dispense the same small dose every time. The feeder also gives the computer vision work something it did not have before: an exact timestamp for every feeding event, logged automatically, which becomes ground truth for behavior classification.

Three pieces work together:

- **Vision Pi.** A Raspberry Pi 5 with an Arducam 12 MP autofocus USB camera (IMX708 sensor) runs a Flask/Waitress MJPEG livestream and local YOLOv8 fish detection with bounding boxes drawn on the frames.
- **Feeder Pi.** A second Raspberry Pi 5, mounted in a printed saddle bracket that hangs off the back of the tank, drives the stepper feeder and serves a small Flask page so anyone on the lab's private Tailscale network can tap "Feed" from a phone.
- **The feeder itself.** A drum feeder with a fixed-volume dose pocket, turned by a 28BYJ-48 geared stepper through a ULN2003 driver board, powered straight from the Pi's 5 V header. No soldering anywhere in the build.

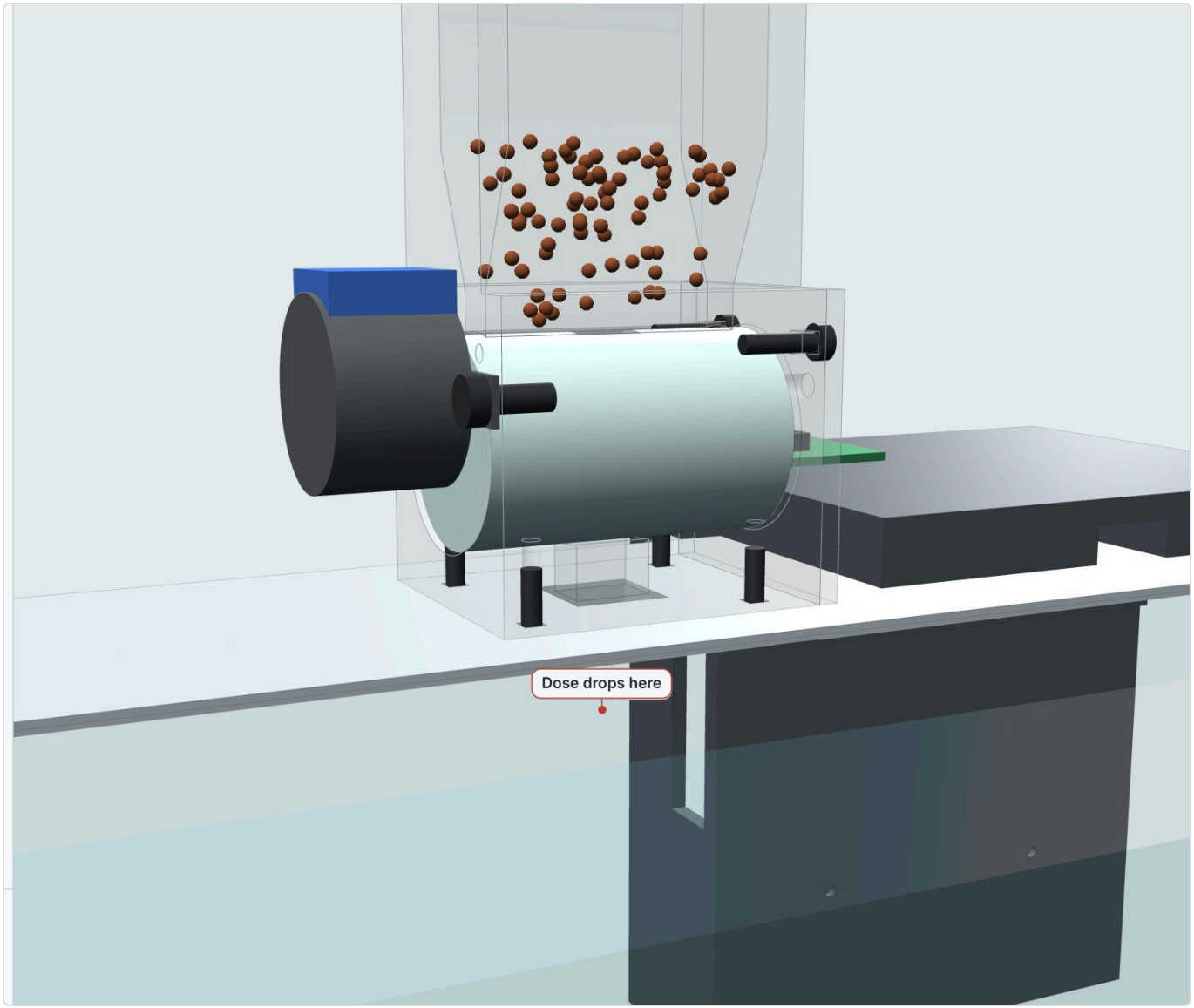
Hardware overview

Item	Detail
Tank	LANDEN 60P, 600 x 300 x 360 mm, 6 mm glass, Seachem Tidal 35 filter
Fish	8 cherry barbs (*Puntius titteya*)
Water testing	API freshwater liquid test kit
Vision computer	Raspberry Pi 5, Arducam 12 MP autofocus USB camera (IMX708)
Feeder computer	Raspberry Pi 5, Raspberry Pi OS Lite 64-bit, booting from a 64 GB USB stick
Motor	28BYJ-48 5 V geared stepper (ELEGOO kit)
Driver	ULN2003 breakout board, 6 female-to-female jumper wires to the Pi header
Printed parts	Cover plate, feeder housing with hopper, drum, end cap, axle pin, saddle, ramp inserts, hopper lid
Fasteners	M3 x 12 (self-tapping, 8), M4 x 10 or M3 x 12 with shim (motor ears, 2), M2.5 x 10 with 5 mm nylon spacers (Pi, 4), M2.5 x 8 (driver board, 2)
Tools	2, 2.5 and 3 mm hex keys, CA glue
Remote access	SSH and Tailscale (double NAT behind the university network, so only outbound tunnels work)

The feeder mechanism

The feeder is a drum feeder. A horizontal cylinder (the drum) sits in a bore inside a printed housing. The drum has one small square pocket cut into its surface. At rest the pocket faces up, directly under a slot in the hopper, so it stays full of pellets.

One feed is one full rotation of the drum. The pocket rotates down, dumps its pellets through a chute in the housing floor and a matching 15 mm hole in the rear cover plate, straight into the water, then continues around and comes back to the top already refilled. Dose size is set by the pocket volume, so every feed is the same amount.



Key dimensions:

- Drum: 31.2 mm diameter, 48 mm long, 0.4 mm running clearance in the bore
- Dose pocket: 6 x 6 mm, roughly 0.09 g of micro pellets per feed (target intake for eight small barbs is 0.15 to 0.3 g per day, so two feeds a day)
- Drop hole in the plate: 15 mm
- Motor to drum: the 28BYJ-48 shaft plugs directly into a D-shaped slot in the drum. The motor's shaft is offset 8 mm from its body center, so the bore sits at Z = 26 mm to keep the motor body clear of the plate.
- Cover plate: 420 x 68 x 4 mm PETG, with a step underneath that sits on the rear glass and a slot for the heater cord

The whole assembly lifts off the tank as a single unit by the plate.

3D design and printing

Parametric CAD in KCL

Every printed part was written as code in KCL for Zoo Design Studio. Writing geometry as code made it easy to change one dimension (drum diameter, pocket size, screw pilot diameter) and regenerate every dependent feature.

Design history:

- **v1:** five loose bodies (housing, drum, end cap, axle pin, separate hopper) in one file, laid out side by side on the bed so nothing printed nested.
- **v2:** hopper fused to the housing (it kept coming loose), axle pin given a head, every joint switched from clearance holes to self-tapping pilot holes so the whole thing screws together and apart with hex keys. Consolidated to three files: `feeder.kcl`, `saddle.kcl`, `plate.kcl`.
- **Post-hardware revisions:** `drum.kcl` with a counterbore and collar relief for the motor's stepped shaft, `ramp.kcl` inserts for the hopper slot, a hopper lid, and a smaller dose pocket.

The STLs were sanity-checked in Blender (scene units set to millimeters, "separate by loose parts" to confirm the body count) and sliced in Bambu Studio.

Geometry checking before printing

Rather than discover collisions on the printer, the KCL files were parsed in Python and rebuilt as real solids with `manifold3d` and `trimesh`. The checker confirmed each body was a single connected piece, intersected bodies pairwise to find collisions, and reported bounding boxes for fit. It caught four real problems before any plastic was used:

1. The Pi board poked 3.5 mm through the floor of the saddle pocket
2. The side window for the Pi's ports was misaligned
3. The motor's wire box collided with the plate
4. Wall thickness around the M4 pilot holes was too thin

```
pip install manifold3d trimesh
```

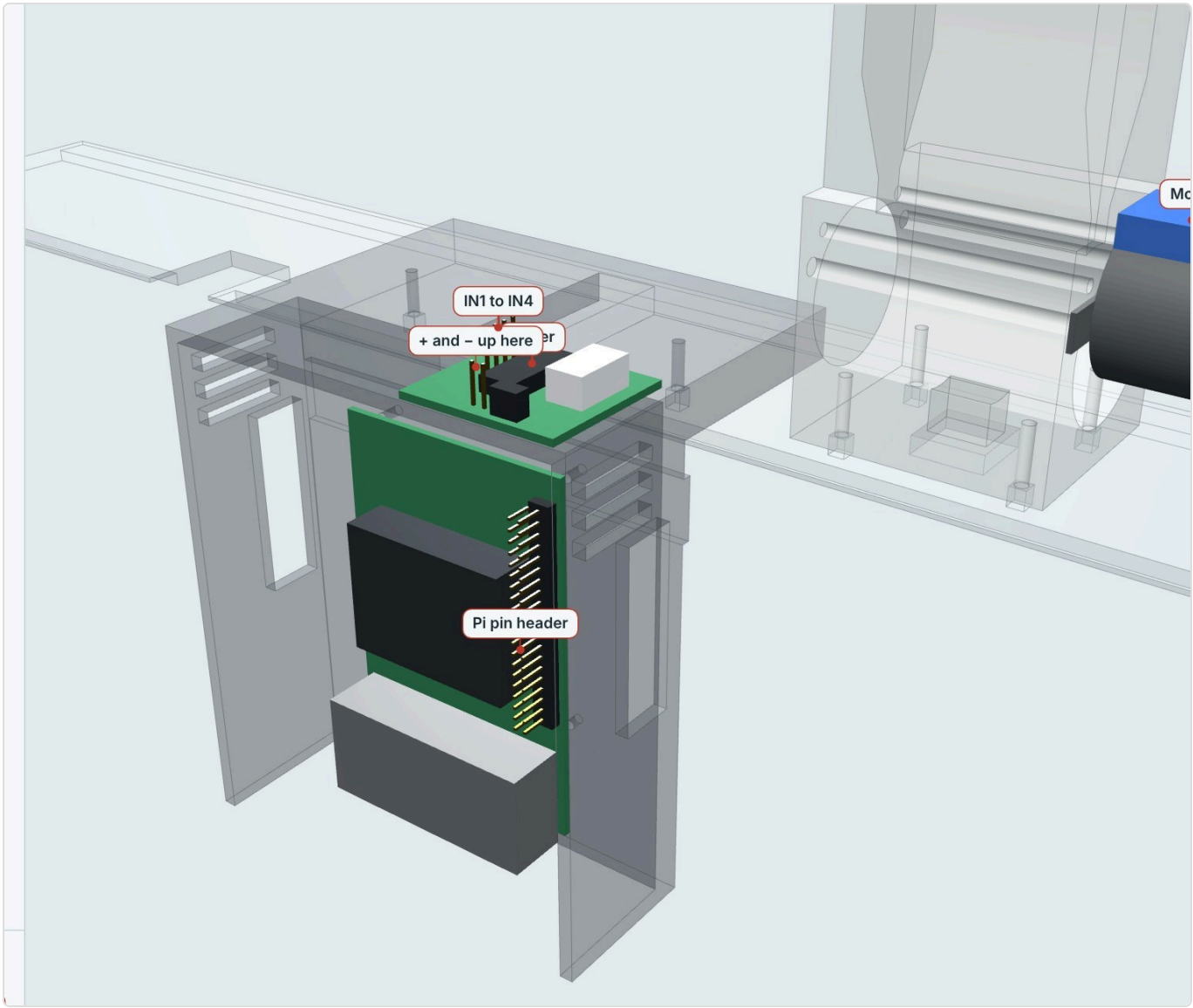
Printing

All parts in PETG at 0.20 mm layers at the i² Studio (Fascitelli Center, URI; Bambu P1S and H2D, Formlabs Form 3, Elegoo Orange Storm Giga for the 420 mm plate).

Part	Orientation	Supports
Housing + hopper	Standing on the motor end, bore vertical	Tree supports under the hopper
Drum	Standing on end	None
End cap, pin, ramps	Flat	None
Saddle	Bridge flat on the bed	None
Plate	Flat (Giga bed)	None

Electronics and wiring

The 28BYJ-48 plugs into the ULN2003 driver with its own 5-pin JST connector. Six jumpers run from the driver to the Pi's 40-pin header. The driver board is mounted on the back half of the saddle, above the Pi pocket and behind the glass line, with two M2.5 x 8 screws straight into the plastic.



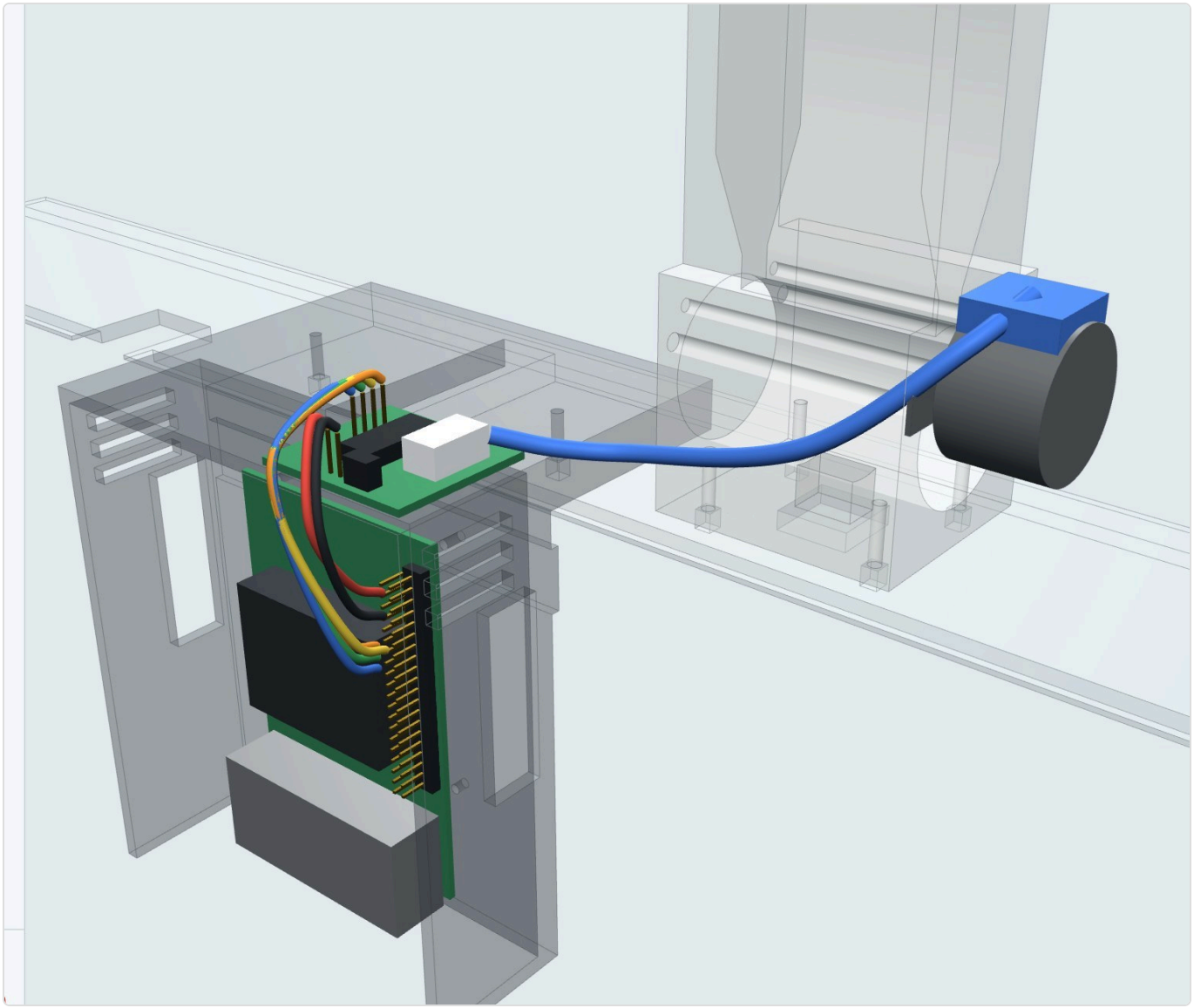
Driver pin	Pi physical pin	Signal
−	6	GND
+	2	5 V
IN1	11	GPIO 17
IN2	12	GPIO 18
IN3	13	GPIO 27
IN4	15	GPIO 22

Pin 1 is the corner pin at the end of the header away from the USB ports. Even pins run down the outer row nearest the board edge, odd pins down the inner row. Swapping + and − can destroy the driver, so check twice before power.

Quick wiring test: each driver LED lights in turn, so a wrong wire is obvious.

```
from gpiozero import OutputDevice
from time import sleep
pins = [OutputDevice(p) for p in (17, 18, 27, 22)]
for i, p in enumerate(pins):
    print(f"IN{i+1} on"); p.on(); sleep(1); p.off()
```

If it buzzes but does not turn, two IN wires are swapped. If nothing happens, check + and −.



Software

`feeder.py` (interactive feeder control)

Pure `gpiozero`, no extra packages. Runs on the feeder Pi over SSH.

- **Half-step drive.** 8-step sequence, `REV = 4076` half-steps per drum turn. The 28BYJ-48's real gear ratio is 63.68:1, not the 64:1 in most tutorials; using 4096 would drift a few degrees every feed.
- **Speed.** 2.5 ms per step with a slow start ramp. The first version ran at 1.5 ms and skipped steps under load, so the drum came back short of home. A feed takes about 12 seconds.
- **Position tracking.** Every step is counted and the phase and position are saved to `~/.feeder_state.json`, so the drum returns to its saved home after quits, reboots and power cuts. Position 0 means the pocket is at the top under the hopper slot. Before every feed the script homes first: if the pocket had not yet reached the drop it backs up (food stays in the pocket); if it was past the drop it continues forward and refills.

- **Daily limit.** `MAX_PER_DAY = 2`, `MIN_GAP_HOURS = 4`, read from `~/feed.log` (one line per feed). `force` overrides it.
- **LED animations.** Scanner sweep, heartbeat at the drop, breathing glow on the driver's four LEDs, plus a fish swimming across the terminal.

Commands at the prompt: **Enter** feeds one dose, `n` / `b` nudge forward or back by 1/64 turn, `home` sets the current position as home, `force` feeds past the daily limit, `q` quits with the motor off and state saved.

Calibration: run the script, look down the hopper, nudge until the pocket is centered under the slot, type `home`. Put a tape flag on the pin head, `force` three feeds and confirm the flag returns to the same spot. Short by a few degrees means skipped steps (slow the delay down). Off by a quarter turn or more means the drum is slipping on the shaft (use the counterbored drum). Delete `feed.log` after testing so test feeds do not count against the fish.

`app.py` (phone feed page)

A small Flask app on the feeder Pi, run as a `systemd` service (`feederweb`) so it survives reboots. The page has a single FEED button wired to a `/feed` route that calls a non-interactive feed, and streams the Arducam live underneath it so you can watch the fish eat. Reached over Tailscale from any phone on the lab tailnet.

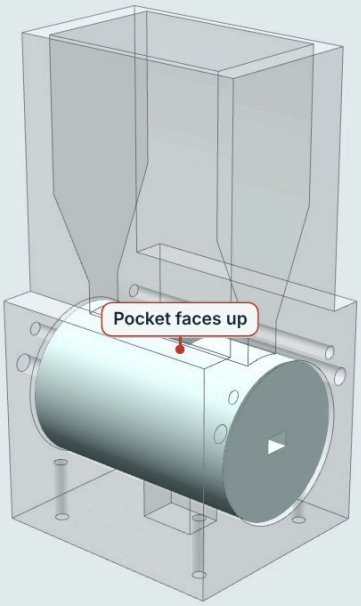
Vision Pi

- 640 x 480 capture, YOLOv8 inference on a 320 x 240 copy, every third frame, to keep the Pi 5 responsive
- Flask/Waitress MJPEG stream with bounding boxes drawn on frames
- Manual focus locked with `v4l2-ctl` so autofocus does not hunt through the glass; a black foam collar pressed to the glass and a slight off-square camera angle kill reflections
- Laplacian-variance blur filtering on fish crops before they go anywhere near a classifier

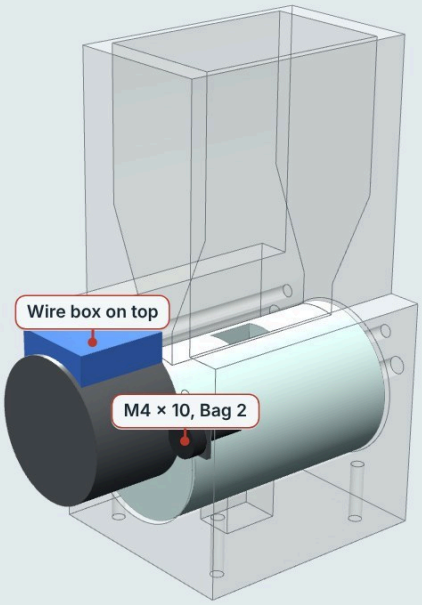
Assembly guide

Every screw self-taps into plastic. Drive each one in, back it out, then drive it in for real and stop at snug.

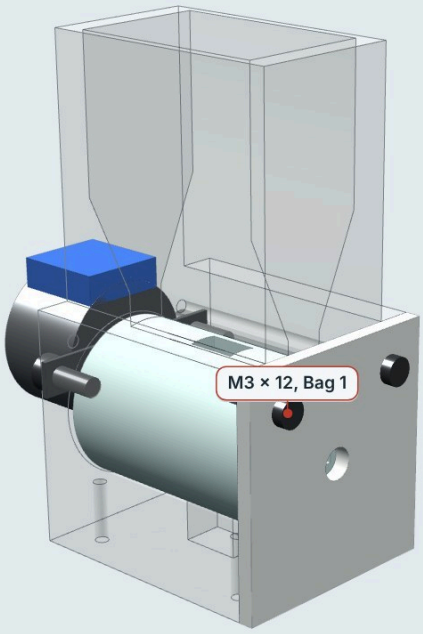
1. Slide the drum into the housing. Pocket facing up. It should spin freely with a small gap all round; if it rubs, sand the drum, not the bore.



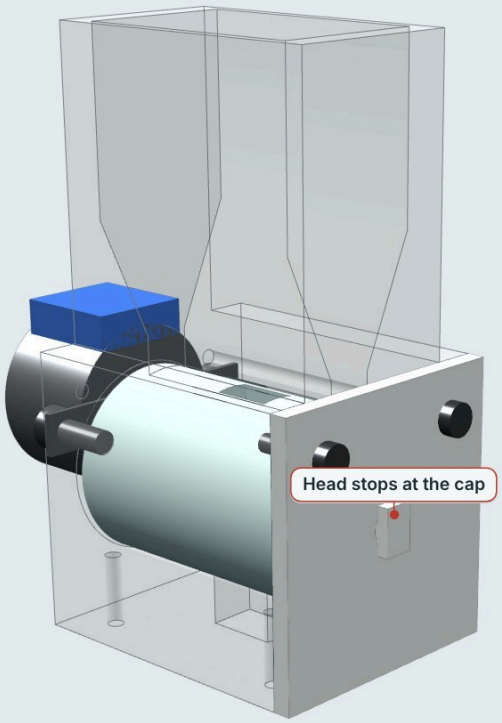
2. Mount the motor on the left end. Turn the drum until its slot lines up with the flats on the motor shaft, push the shaft in with the wire box on top, and screw the ears to the housing. Left end means the end away from the saddle; on the other end the motor body would hit it.



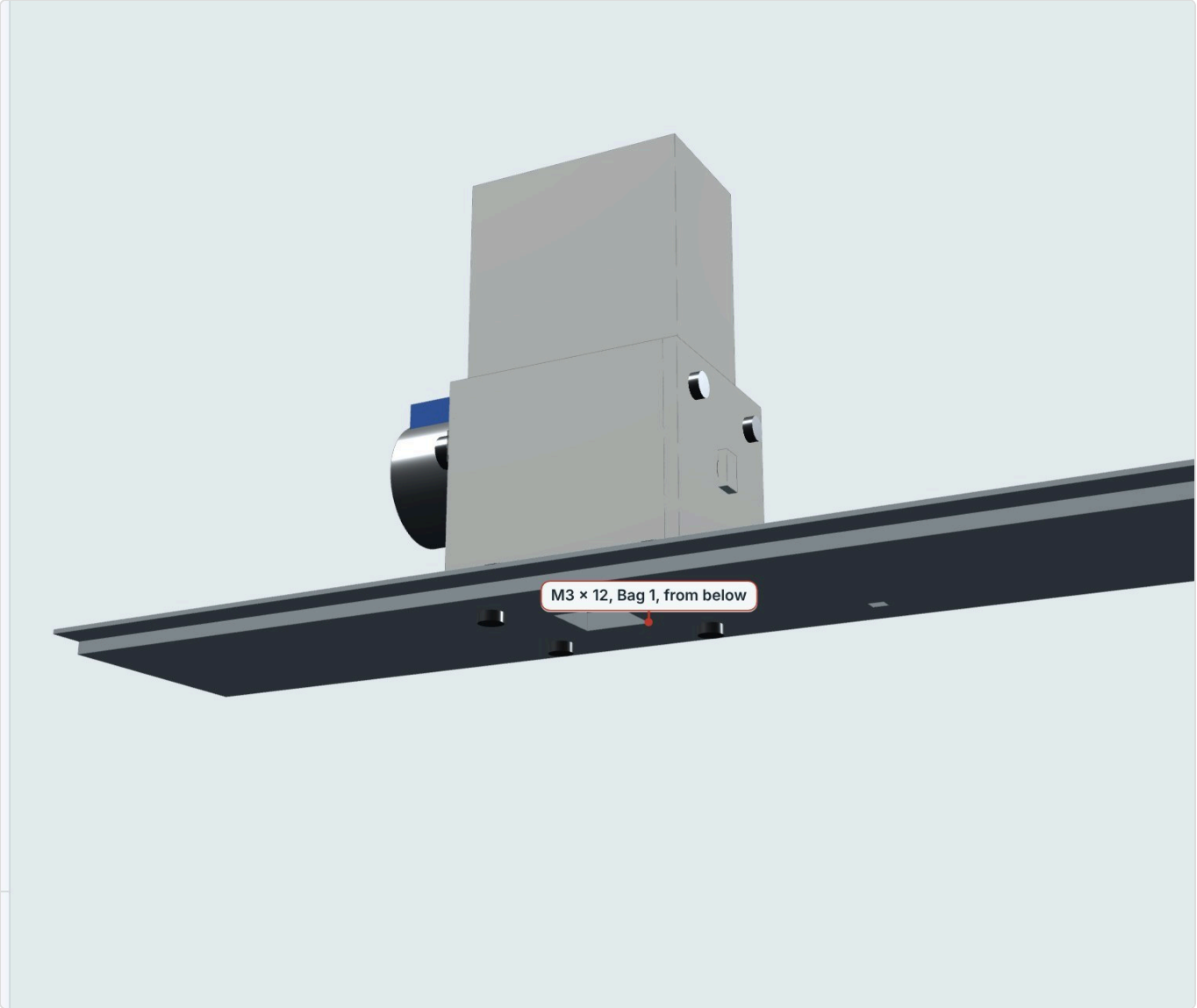
3. Screw on the end cap. Two M3 x 12 into the small holes near the top. The round center hole is for the pin, not a screw.



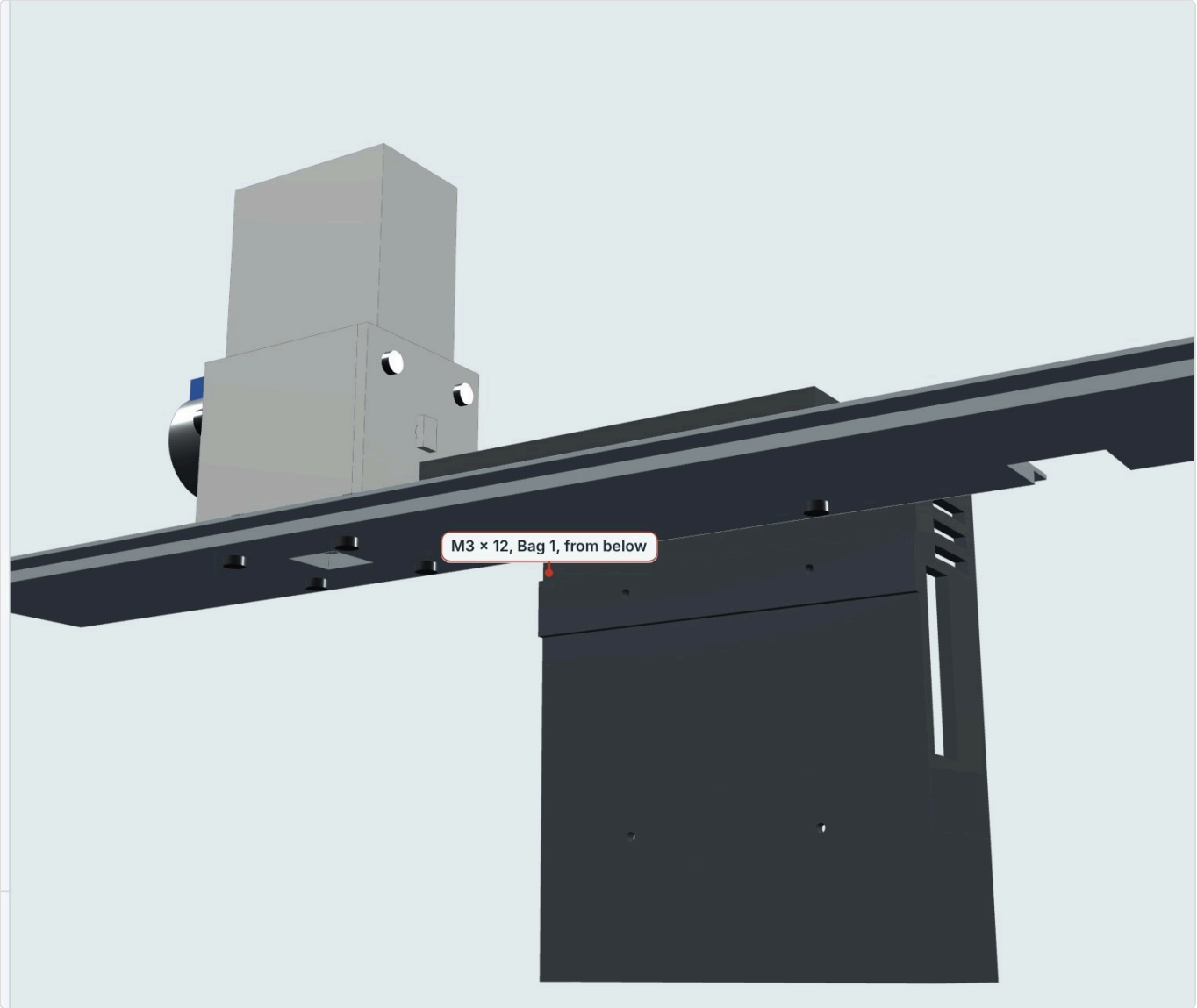
4. Push in the pin. Through the cap's round hole into the drum's slot until the head touches the cap. Drum and pin should turn together. Too loose: one drop of CA glue on the part inside the drum.



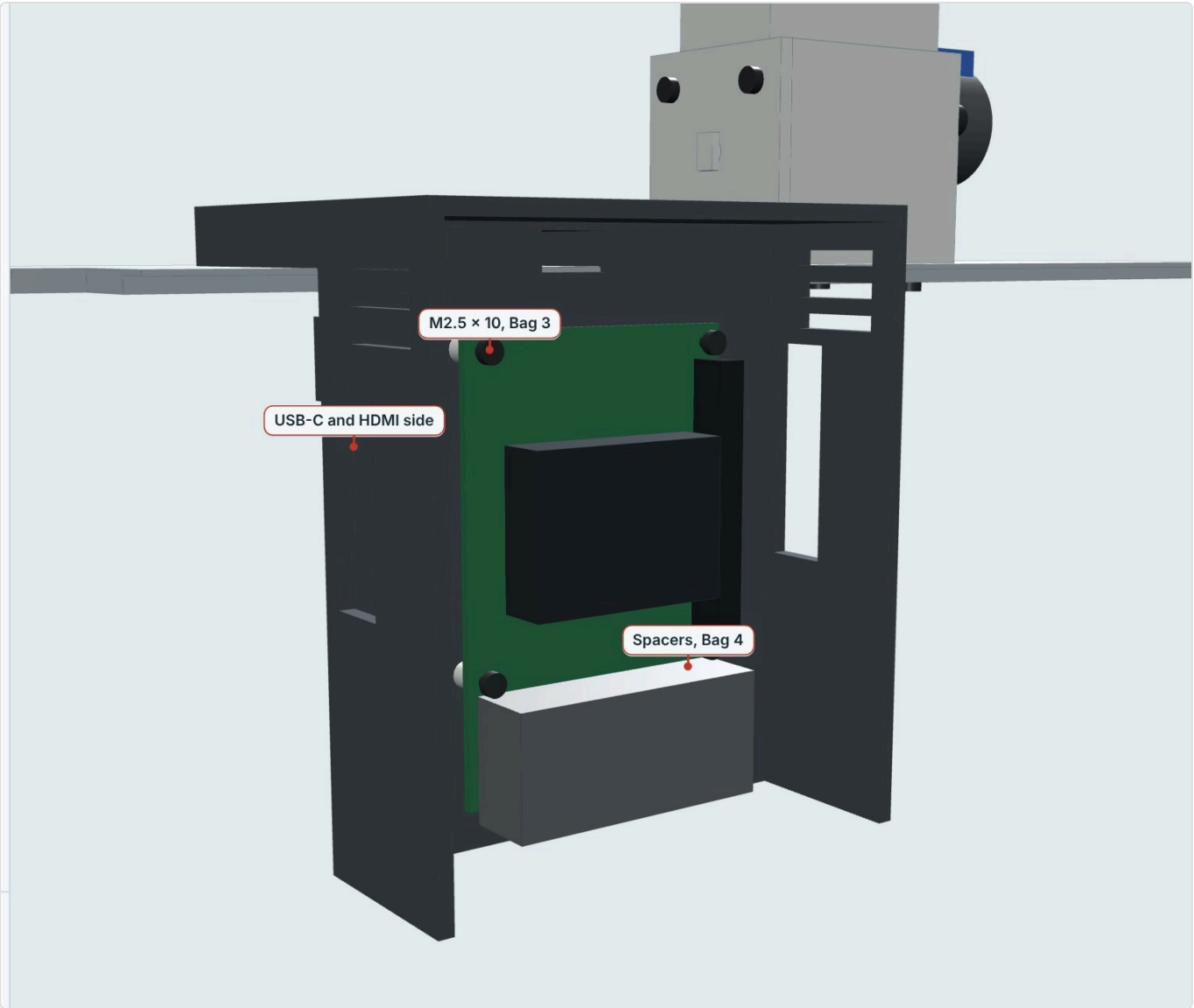
5. Mount the feeder on the plate. With the plate off the tank and over a table edge, set the chute over the plate's square hole and drive four M3 x 12 up from underneath. Snug only; these heads hang under the plate over the water.



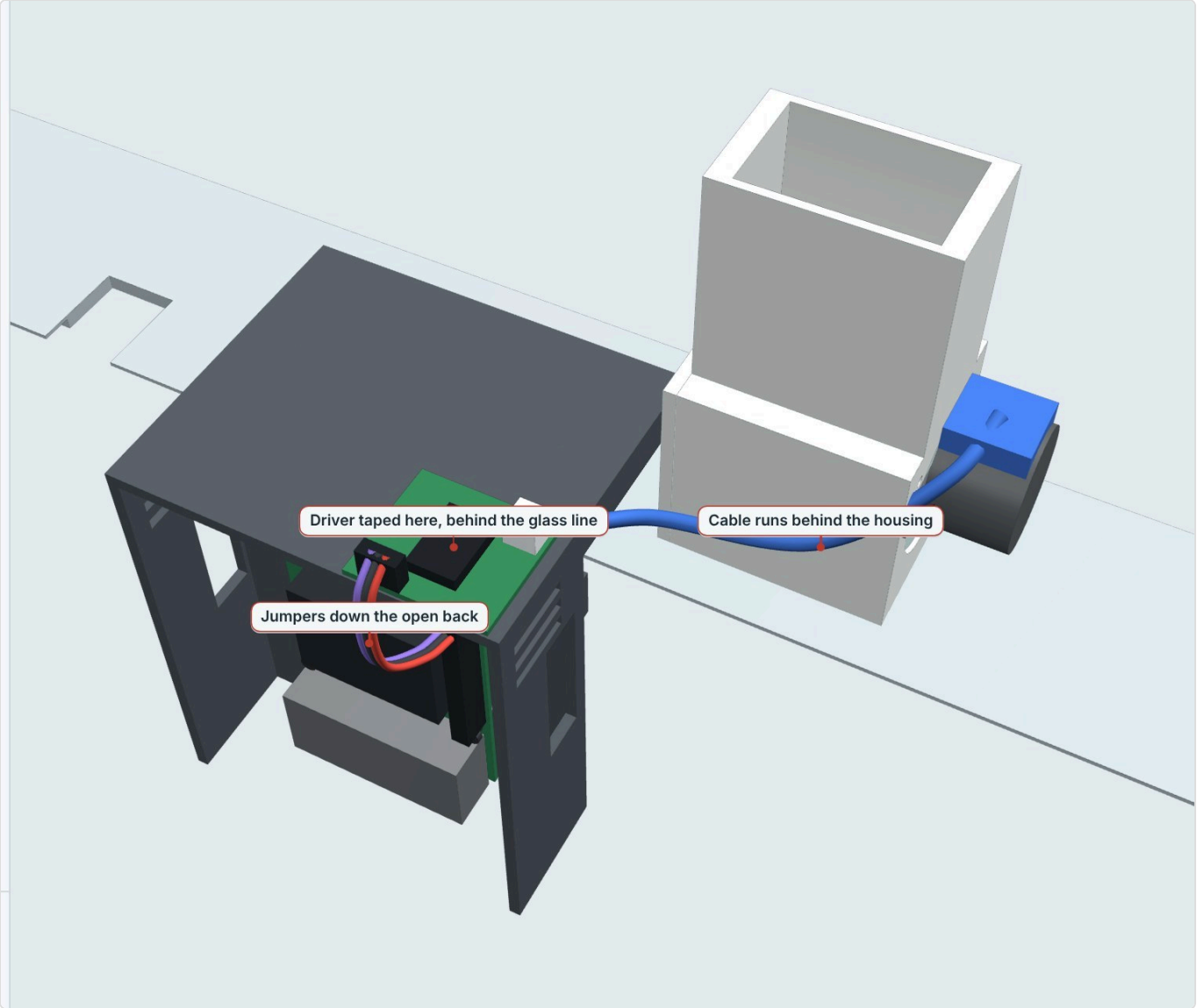
6. Mount the saddle. Flat bridge on the plate, tall leg hanging past the back edge, two M3 x 12 up from below.



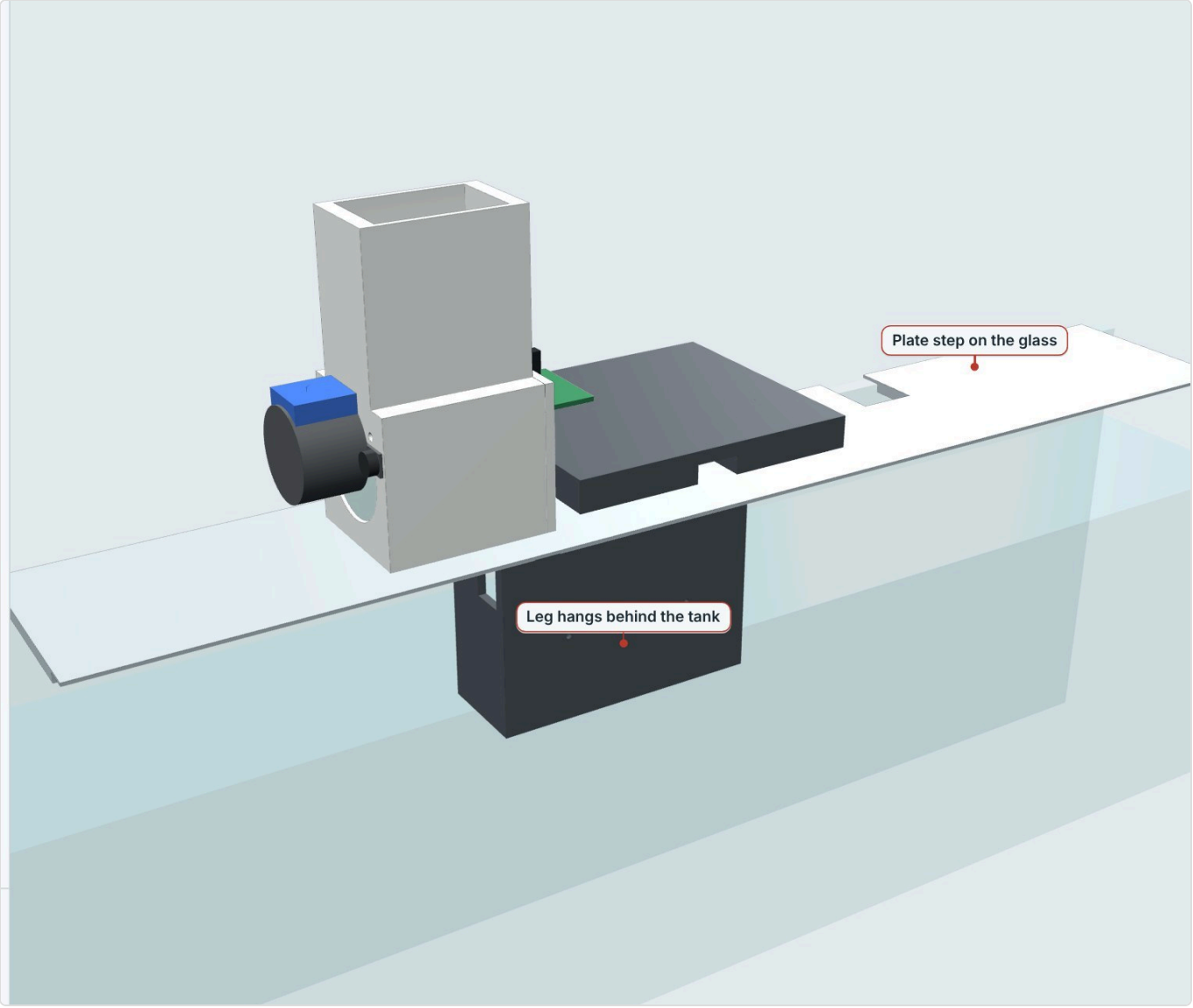
7. Mount the Pi 5. USB and Ethernet face down out the open bottom; USB-C and HDMI face the side window. A 5 mm spacer on each of the four M2.5 x 10 screws. Anything longer than 10 mm pokes through the leg and touches the glass.



8. Wire it up. Driver on the back half of the saddle, motor cable behind the housing, six jumpers down the open back onto the Pi header. Pi off while wiring.



9. Set it on the tank. Lower it straight down onto the back of the tank. The step under the plate sits on the rear glass; the saddle leg and Pi hang behind. Lift by the plate, never by the hopper or the saddle.

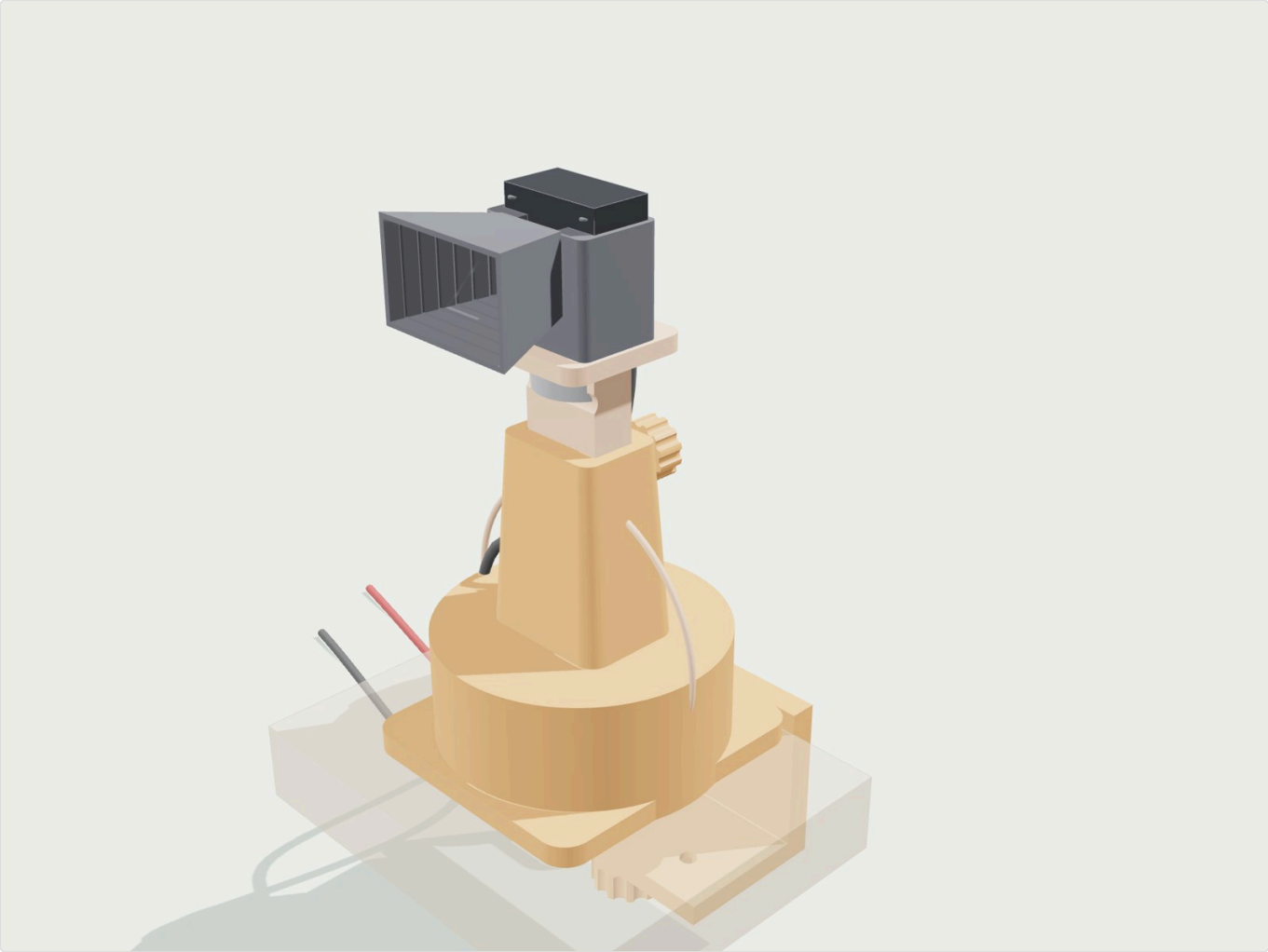


10. Test a feed. Fill the hopper with micro pellets or granules, not flakes (flakes jam the drum). Keep the hopper dry; damp pellets clump.

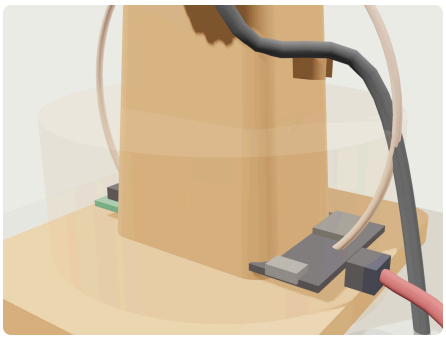
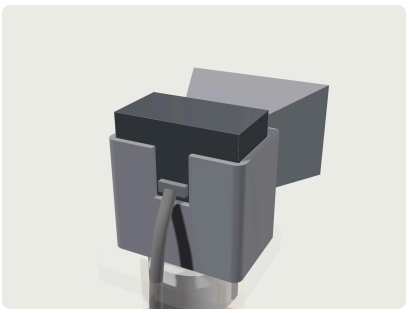
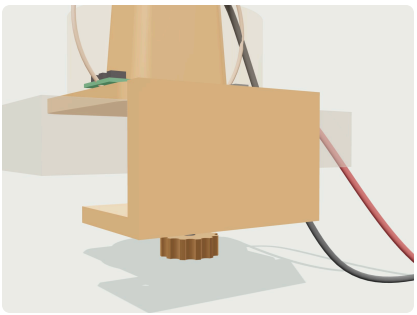
Camera mount (next build)

The Arducam's stock metal stand is too short to see the whole tank, so a printed mount is designed and ready to print (v5, one combined KCL file):

- A clamp base that grips the desk edge sideways instead of standing free
- A telescoping post with a height lock knob
- A slide-in dock the camera cannot shake out of



A motorized pan head is designed for a later revision. A second 28BYJ-48 and driver sit stationary inside an enclosed pod in the base, driven by a spare ESP32 over Wi-Fi and powered by one USB-C cable from the Pi. A shaft runs up the column to a drum the camera rides on, giving about 350 degrees of sweep with a soft stop (the camera's own USB cable is what would otherwise wind up). That lets the same camera pan between the tank and a second robot project on the floor behind it.

Cutaway showing the pod	Dock detail	Desk clamp
		

Machine learning

The tank is the deployment target for an ELE 588 (Applied Machine Learning) course project on detecting unhealthy fish from appearance:

- Classical image features (color histograms, LBP texture) from fish crops
- Decision tree, kNN, SVM (RBF) and MLP classifiers in scikit-learn, trained on the public MatsyaDx-BD freshwater fish disease dataset
- ADASYN oversampling for the rare diseased classes
- Three-tier evaluation: held-out dataset test set, healthy cherry barb crops from this tank for false-alarm rate, and barb crops with real lesion patches composited on as a sensitivity check

YOLO on the Pi handles detection and cropping; the classifier runs on full-resolution stills so each fish crop stays above the roughly 100 px minimum the features need. The feeder's `feed.log` doubles as labeled feeding events for a future behavior classifier (dark / fed / normal) trained on the tank's own logs.

What went wrong and what fixed it

Almost every real problem came from the gap between the datasheet and the part in hand.

Problem	Fix
Zoo `union` is unsupported for bodies that share only flat faces	Draw one closed multi-segment outline and extrude once
Zoo boolean subtract fails when a cutter face lands flush with an existing face	Every cutter face must sit in empty space or create a new plane: overshoot into air or offset by 0.1 mm, use one L-shaped cutter, chain cuts one per step so the error points at a single cut
Motor ear holes were too small for M4 screws	M3 x 12 with a filament shim (the print lab only stocks metric)
Drum D-slot did not account for the stepper's stepped shaft	Counterbore and collar relief in `drum.kcl`, verified against the real shaft
Driver board had no mount	Screw-mounted to the saddle back with 2 x M2.5 x 8 into plastic
Dose pocket (6 x 10 mm) held too much for eight small fish	Resized to 6 x 6 mm, confirmed by volume calculation
Pellets crushed in the unused part of the feed slot	Two ramp inserts funnel pellets into the pocket
Axle pin printed loose	Tape for now; CA glue or reprint at corrected diameter
Drum returned short of home	Step delay slowed from 1.5 ms to 2.5 ms; no PWM on coils while stepping
microSD card stalled ("card stuck being busy", I/O errors)	Boot the feeder Pi from a USB stick, boot order set to USB first
Hopper separate from housing kept coming loose	Fused into one body in v2

Status and roadmap

Working

- All parts printed, drum fits the bore, cap and pin installed, motor mounted and turning
- Pi 5 in the saddle, driver wired, homing repeatable at 2.5 ms per step
- `feeder.py` with position memory and daily limits
- Flask feed page with live camera stream, running as a service, reachable over Tailscale
- Arducam on the feeder Pi, USB boot for reliability

Next

- Glue or reprint the axle pin at the corrected size
- Print and install the two ramp inserts and the hopper lid
- Scheduled feeds (systemd timer calling a non-interactive feed)
- Pipe `feed.log` timestamps into the vision data collection as feeding-event labels
- Print the camera mount v5 (measure desk thickness and camera case first), then add the pan head

- ESP32 temperature sensor as a second data source on the dashboard
- Longer term: a gutted transparent LCD on the front glass showing live YOLO overlays

Repository layout

```
cad/
  feeder.kcl        housing with fused hopper, drum, end cap, axle pin
  drum.kcl          counterbored drum for the stepped 28BYJ-48 shaft
  ramp.kcl          hopper slot ramp inserts
  saddle.kcl        Pi 5 bracket that hooks over the rear plate
  plate.kcl         420 mm rear cover plate with drop hole
  camera_mount.kcl  desk clamp, telescoping post, camera dock (v5)
  stl/              exported meshes as printed
tools/
  geometry_check.py manifold3d + trimesh collision and fit checker
feeder/
  feeder.py         interactive stepper control
  feed_test.py      wiring test
  app.py            Flask feed page with camera stream
  feederweb.service systemd unit
vision/
  main.py           MJPEG stream and YOLOv8 detection
docs/
  images/           renders from the build and wiring guides
  photos/           photos of the real build
```

Secrets (Wi-Fi credentials, Tailscale node addresses, tokens) are intentionally not in this repository.

Acknowledgements

Built in Dr. Yan Sun's CYPHER Lab at the University of Rhode Island. Parts printed by the i² Studio at the Fascitelli Center for Advanced Engineering.